Laboratório de Desenolvimento de Software - LABORATÓRIO 3

CONQUISTA

Sistema de moeda estudantil construido com React, FastAPI, SQLAlchemy e PostgreSQL

Integrantes: Arthur Martins, Felipe Costa, Gustavo Leonardi e Sofia Fernandes

TECNOLOGIAS UTILIZADAS E ORGANIZAÇÃO

FRONTEND

React + TypeScript
Vite

BACKEND

FastAPI
Pydantic

BANCO

PostgreSQL
Neon + Alembic

PERSISTÊNCIA

SQLAlchemy
Repository

MVC

View: React
Controller: FastAPI Router
Model: entidades e dados com SQLAlchemy

Camadas adicionais

Regras de negócio no Service e consultas no repository

Comunicação

Frontend e backend trocam dados por HTTP, em formato JSON.

1 - REACT + TypeScript

Exemplo no sistema

Arquivo: frontend/src/pages/AlunoForm.tsx

```
const [inst, setInst] = useState<Instituicao[]>([]);

const {
  register,
  handleSubmit,
  reset,
  formState: { errors, isSubmitting }
} = useForm<Form>({
  resolver: zodResolver(schema)
});
```

useState

Armazena dados que mudam na interface. Nesse exemplo, a lista de instituições.

React Hook Form

Gerencia campos, envio e estado de validação do formulário.

TypeScript

Define tipos, como Instituicao e Form, para reduzir erros no desenvolvimento.

2 - VALIDAÇÃO DE DADOS: FRONT E BACK

No FrontEnd com Zod

Arquivo: frontend/src/pages/AlunoForm.tsx

```
const schema = z.object({
 nome: z.string().min(2),
 email: z.string().email(),
 instituicao_id: z.coerce.number().positive()
});
```

Validar no frontend melhora a experiência

No BackEnd com Pydantic

Arquivo: backend/app/schemas/aluno.py

```
class AlunoCreate(BaseModel):
 nome: Annotated[str, Field(min_length=2)]
 email: EmailStr
 instituicao_id: int = Field(gt=0)
```

Validar no backend protege os dados e a API independentemente de quem a consome.

3 - FastAPI: Endpoints

Exemplo no sistema

Arquivo: backend/app/routers/api.py

```
@router.get("/alunos", response_model = list[AlunoResponse])
def listar_alunos(db: Db, busca: str | None = Query(None)): return AlunoService(db).list(busca)

@router.post("/alunos", response_model = AlunoResponse, status_code=status.HTTP_201_CREATED)
def criar_aluno(data: AlunoCreate, db: Db): return AlunoService(db).create(data)

@router.put("/alunos/{entity_id}", response_model=AlunoResponse)
def atualizar_aluno(entity_id: int, data: AlunoUpdate, db: Db): return AlunoService(db).update(entity_id, data)

@router.delete("/alunos/{entity_id}", status_code=status.HTTP_204_NO_CONTENT)
def excluir_aluno(entity_id: int, db: Db): AlunoService(db).delete(entity_id); return Response(status_code=204)
```

GET e GET /alunos

Consulta e lista alunos e recebe dados para cadastrar um aluno.

PUT E DELETE /alunos/{entity_id}

Atualização e exclusão também estão declaradas no router.

response_model

Define o formato da resposta

4 - SERVICE COM FastAPI

O Router recebe a requisição e o Service decide o que precisa acontecer.

Exemplo no sistema

Arquivo: backend/app/routers/api.py (ADAPTADO)

```
def create(self, data: AlunoCreate) -> AlunoResponse:
 if not self.repo.instituicao(data.instituicao_id): raise NotFoundError("Instituição não encontrada")
 if self.repo.email_exists(str(data.email)): raise ConflictError("E-mail já cadastrado")

 self.db.add(user)
 self._commit()
```

1. Verificação

Confere se a instituição informada existe.

2. Conferir duplicidade

Verifica se o e-mail já está cadastrado.

3. Se tiver tudo certo, persiste

Adiciona os objetos à sessão e confirma o envio.

5 - REPOSITORY COM SQLAlchemy

SQLAlchemy traduz operações Python para SQL para fazer as buscas dos dados que vão ser enviados pro front

Exemplo no sistema

Arquivo: backend/app/repositories/usuario.py

```
stmt = select(Usuario.id).where(Usuario.email == email.lower())
if except_id is not None:
  stmt = stmt.where(Usuario.id != except_id)
return self.db.scalar(stmt) is not None
```

SQL Equivalente

```
SELECT id
FROM usuarios
WHERE email = LOWER(email);
```

6 - MODELS EM PYTHON

Exemplo no sistema

Arquivo: backend/app/models/entities.py

```
class Aluno(Base): __tablename__ = "alunos"

 usuario_id: Mapped[int] = mapped_column (
   ForeignKey("usuarios.id"), primary_key=True
 )
 cpf: Mapped[str] = mapped_column (
   String(11), unique=True, nullable=False
 )
```

Base / __tablename__

Identifica a classe mapeada e o nome da tabela.

Mapped / mapped_column

Declara atributos e colunas.

ForeignKey

Relaciona Aluno com Usuario.

7 - PostgreSQL, Neon e Alembic

PostgreSQL

Banco de dados relacional que armazena os registros.

Neon

Serviço de PostgreSQL hospedado na nuvem.

Alembic

Ferramenta de versionamento das mudanças no schema.

Exemplo no sistema

Arquivo: backend/alembic/versions/20261026_0001_sprint2.py (ADAPTADO_

```
def upgrade():
    op.create_table("instituicoes_ensino", ...)

def downgrade():
    op.drop_table("instituicoes_ensino")
```

upgrade() aplica a alteração e downgrade() desfaz a revisão.
SQLAlchemy mapeia os objetos e Alembic versiona a estrutura do banco.

8 - Axios - FRONTEND COM A API

A separação facilita reutilizar chamadas e manter a comunicação HTTP organizada.

Exemplo no sistema

Arquivo: frontend/src/services/alunoService.ts

```
export const alunoService = {
  listar: async () =>
    (await api.get<Aluno[]>('/alunos')).data,

  criar: async (data: AlunoCreate) =>
    (await api.post<Aluno>('/alunos', data)).data,

  atualizar: async (id, data) =>
    (await api.put(`/alunos/${id}`, data)).data
};
```

Serviço centralizado

A página chama alunoService, em vez de concentrar HTTP no componente.

api.ts

Configura a instância Axios e a URL base.

9 - ARQUITETURA DO CONQUISTA

A separação facilita reutilizar chamadas e manter a comunicação HTTP organizada



10 - MODELO ER E PERSISTÊNCIA

Entidades centrais da Sprint 3 e relacionamento



Persistência e Integridade

SQLAlchemy: ORM | Alembic : migrações | PostgreSQL: chaves e restrições